بسم الله الرحمن الرحیم

دانشگاه صنعتی شریف
دانشکده‌ی مهندسی کامپیوتر

یادگیری عمیق

نیم‌سال دوم ۰۵-۰۴

مدرس: مهدیه سلیمانی

تمرین دوم | ددلاین تمرین: ۸ خرداد

- برای ارسال هر تمرین تا ساعت ۲۳:۵۹ روز ددلاین فرصت دارید. مهلت تاخیر (مجاز و غیر مجاز) برای این تمرین ۷ روز است (یعنی حداکثر تاریخ ارسال تمرین ۱۵ خرداد است)
- در هر کدام از سوالات، اگر از منابع خارجی استفاده کرده‌اید باید آن را ذکر کنید. در صورت همفکری با افراد دیگر هم باید نام ایشان را در سوال مورد نظر ذکر نمایید.
- پاسخ تمرین باید ماحصل دانسته‌های خود شما باشد. در صورت رعایت این موضوع، استفاده از ابزارهای هوش مصنوعی با ذکر نحوه و مصداق استفاده بلامانع است.
- پاسخ ارسالی واضح و خوانا باشد. در غیر این صورت ممکن است منجر به از دست دادن نمره شود.
- از تحویل تمارین نظری به شکل تایپ شده (فایل Word-Latex یا هر نوع تایپ شده دیگری) بپرهیزید و حتما پاسخ دست نویس تحویل دهید.
- پاسخ ارسالی باید توسط خود شما نوشته شده باشد. به اسکرین‌شات از منابع یا پاسخ افراد دیگر نمره‌ای تعلق نمی‌گیرد.
- در صورتی که بخشی از سوال‌ها را جای دیگری آپلود کرده و لینک آن را قرار داده باشید، حتما باید تاریخ آپلود مشخص و قابل اتکا باشد.
- محل بارگذاری سوالات نظری و عملی در هر تمرین مجزا خواهد بود. به منظور بارگذاری بایستی تمارین تئوری در یک فایل pdf با نام HW1_[First-Name]_[Last-Name]_[Student-Id].pdf بارگذاری شوند.
- در صورت وجود هرگونه ابهام یا مشکل، در کوئرای درس آن مشکل را بیان کنید و از پیغام دادن مستقیم به دستیاران آموزشی خودداری کنید.
- طراحان این تمرین : علیرضا علایی، مبین شریفی‌راد، عرفان صبحائی، شایگان ادیم، امیرحسین ایزدی، امیرحسین کوچکیان، محمد امانلو، عرفان یگانگی دستگردی، حسین شاه‌آبادی

بخش نظری (۱۰۰+۱۰ نمره)

پرسش ۱. عملگر کانولوش (۱۵ نمره)

فرض کنید $I \in \mathbb{R}^{H \times W}$ یک تصویر و عملگر انتقال [۱] را به صورت زیر تعریف می‌کنیم:

$$(T_{a,b}I)(i,j) = I(i-a, j-b)$$

همچنین یک عملگر $\mathcal{L}$ را هموردا نسبت به انتقال [۲] گویند اگر برای هر انتقال (a,b) داشته باشیم:

$$\mathcal{L}(T_{a,b}I) = T_{a,b}(\mathcal{L}(I)).$$

[۱] Translation Operator
[۲] Translation Equivariant

همچنین کانولوشن با هسته $K \in \mathbb{R}^{k\times k}$ به صورت زیر تعریف می‌شود:

$$(K * I)(i,j) = \sum_{m,n} K(m,n)\, I(i-m, j-n).$$

آ) نشان دهید کانولوشن نسبت به انتقال هموردا است.

ب) ثابت کنید هر عملگر خطی که نسبت به انتقال هموردا است، یک کانولوشن است. توضیح دهید این نتیجه چه تاثیری بر انتخاب شبکه‌های کانولوشنی و طراحی آن‌ها دارد.

ج) فرض کنید تصاویر قبل ورود به شبکه به بردار $x \in \mathbb{R}^{HW}$ تبدیل می‌شوند.
ابتدا نشان دهید که ماتریس W متناظر با تبدیل خطی که نسبت به انتقال هموردا است، یک ماتریس Toeplitz است. سپس توضیح دهید چرا یک لایه خطی موجود در MLP به طور کلی نمی‌تواند نسبت به انتقال هموردا باشد.

د) فرض کنید از یک شبکه عصبی دو لایه برای تشخیص یک الگو در یک تصویر استفاده می‌کنیم. حداقل تعداد پارامتر قابل یادگیری مورد نیاز در لایه اول برای تشخیص یک الگوی $k \times k$ را به ازای لایه اول کانولوشنی و خطی کاملا متصل محاسبه و مقایسه کنید.

تعریف: ماتریس Toeplitz، ماتریسی است که درایه‌های هر قطر موازی با قطر اصلی، با هم برابر باشند. به عبارت دیگر مقدار درایه‌ها تابعی از اختلاف شماره سطر و ستون آن است. برای مثال ماتریس T که به صورت زیر تعریف شده، یک ماتریس Toeplitz است.

$$T = \begin{bmatrix} T_0 & T_{-1} & T_{-2} & T_{-3} \\ T_1 & T_0 & T_{-1} & T_{-2} \\ T_2 & T_1 & T_0 & T_{-1} \\ T_3 & T_2 & T_1 & T_0 \end{bmatrix}$$

پاسخ. ◁

آ)

$$\begin{aligned}
(K * T_{a,b}I)(i,j) &= \sum_{m,n} K(m,n)\,(T_{a,b}I)(i-m, j-n) \\
&= \sum_{m,n} K(m,n)\, I(i-m-a, j-n-b) \\
&= \sum_{m,n} K(m,n)\, I(k-m, l-n) \\
&= (K * I)(k,l) = (K * I)(i-a, j-b) \\
&= T_{a,b}(K * I)(i,j)
\end{aligned}$$

ب) فرض می‌کنیم $\delta_{i,j}$ تصویری است که تنها درایه (i,j) آن یک و باقی درایه‌ها صفر است. در واقع این تصویر یک سیگنال ضربه دو بعدی است.
توجه داریم که

$$\delta_{i,j} = T_{i,j}\delta_{0,0}.$$

همچنین یک تصویر دلخواه را به شکل مجموع این تصاویر نوشت:

$$I = \sum_{i,j} I(i,j)\delta_{i,j}$$

حالا تبدیل خطی $\mathcal{L}$ را در نظر می‌گیریم و تعریف می‌کنیم:

$$\mathcal{L}(\delta_{0,0}) = K$$

در واقع در اینجا پاسخ تبدیل خطی به ضربه را تعریف کردیم. حالا نشان می‌دهیم اعمال این تبدیل خطی همان کانولوشن است.

$$\begin{aligned}
\mathcal{L}(I) &= \mathcal{L}(\sum_{i,j} I(i,j)\delta_{i,j}) \\
&= \sum_{i,j} I(i,j)\mathcal{L}(\delta_{i,j}) \\
&= \sum_{i,j} I(i,j)\mathcal{L}(T_{i,j}\delta_{0,0}) \\
&= \sum_{i,j} I(i,j)T_{i,j}(\mathcal{L}(\delta_{0,0})) \\
&= \sum_{i,j} I(i,j)T_{i,j}K \\
&= K * I
\end{aligned}$$

توجه شود که در گام آخر از این تعریف کانولوشن استفاده شده است:

$$(K * I)(i,j) = \sum_{m,n} K(i-m, j-n)\, I(m,n) = \sum_{m,n} (T_{m,n}K)(i,j)\, I(m,n)$$

بنابراین نشان دادیم هر تبدیل خطی هموردا نسبت به انتقال قابل نمایش به فرم کانولوشن است.

این موضوع باعث می‌شود انتخاب لایه‌های کانولوشنی به یک شرط لازم و کافی برای shift equivariant بودن تبدیل شود. توجه شود مطابق الگو معمول شبکه‌های عصبی، در ابتدا به دنبال یک تبدیل خطی هستیم. این تبدیل خطی همان کانولوشن است. در مرحله بعد می‌توان تبدیل غیر خطی را بر روی خروجی لایه کانولوشن اعمال کرد.. این تبدیل خاصیت هموردا بودن نسبت به انتقال را حفظ می‌کند. بنابراین می‌توان توالی از لایه‌های کانولوشنی و تبدیلات غیرخطی را داشت و به شبکه‌های عمیق کانولوشنی رسید. البته توجه شود در این بررسی stride برابر با یک فرض شد.

ج) با توجه به بخش قبل می‌دانیم این اپراتور خطی همان کانولوشن است. همچنین می‌دانیم کانولوشن نسبت به انتقال هموردا است. بنابراین ماتریس انتقال و ماتریس کانولوشن باید قابل جابجایی با یکدیگر باشند. یعنی $TW = WT$.

با توجه به توضیحات مسئله، در این بخش ورودی‌ها از جنس بردار هستند و با توجه به ابعاد محدود بردارها، شفیت دایره‌ای مد نظر خواهد بود. یک واحد شیفت دادن در این صورت با ضرب ماتریس زیر اتفاق می‌افتد.

$$T = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 & 1 \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}, \qquad T^n = I.$$

توجه شود این ماتریس وارون پذیر بوده و وارون آن در خلاف جهت شیفت می‌دهد. همچنین بررسی جابجاپذیر بودن این ماتریس با ماتریس کانولوشن برای در نظر گرفتن تمام حالات شیفت کافی است زیرا باقی حالات از ضرب این ماتریس یا وارون آن ایجاد می‌شوند.

شرط درست بودن $(WT)_{i,j} = (TW)_{i,j}$ به صورت $W_{i,j+1} = W_{i-1,j}$ است. بنابر این دسته از شروط، ماتریس W به شکل زیر خواهد بود:

$$W = \begin{bmatrix} w_0 & w_{n-1} & w_{n-2} & \cdots & w_1 \\ w_1 & w_0 & w_{n-1} & \cdots & w_2 \\ w_2 & w_1 & w_0 & \cdots & w_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{n-1} & w_{n-2} & w_{n-3} & \cdots & w_0 \end{bmatrix}.$$

و بنابراین ثابت شد که ماتریس کانولوشن یک ماتریس Toeplitz است. روش دیگر اثبات استفاده مستقیم از تعریف کانولوشن است که در آن برخی از این درایه‌ها صفر می‌شوند.

توجه شود یک لایه خطی به طور کلی به صورت ضرب یک ماتریس عمل می‌کند و شرط خاصی روی ماتریس خود اعمال نمی‌کند. این درحالی است که نشان دادیم ماتریس کانولوشن باید Toeplitz باشد و بنابراین در حالت کلی یک لایه خطی نمی‌تواند هموردا نسبت به انتقال باشد.

د) برای تشخیص یک الگوی خاص در یک تصویر کافی است یک فیلتر با تعداد پارامتر $k \times k$ داشته باشیم و آن را با تصویر اصلی کانوالو کنیم. از طرفی برای تشخیص یک الگو مشخص در MLP مجبور هستیم که تمام مربع‌های $k^2 + 1$ موجود در تصویر را در نظر بگیریم و به یک نورون بدیم. در واقع وزن سایر پیکسل‌ها را صفر کرده‌ایم. با این حال چون شبکه خطی است مجبور به اتصال سایر پیکسل‌ها نیز هستیم. در این حالت به $(HW + 1)(H - k + 1)(W - k + 1)$ پارامتر نیاز است. این تفاوت ناشی از نبود خاصیت parameter sharing است که موجب می‌شود یک لایه خطی مجبور باشد وزن‌های جداگانه‌ای برای هر لوکیشن یاد بگیرد.

پرسش ۲. محاسبات Forward و Backward در شبکه‌های کانولوشنی (۱۵ نمره)

فرض کنید ماتریس ورودی X و کرنل W به صورت زیر داده شده‌اند.

$$X = \begin{bmatrix} 1 & 2 & 0 & 3 & 1 \\ 2 & 1 & 3 & 0 & 2 \\ 0 & 1 & 2 & 1 & 1 \\ 3 & 0 & 1 & 2 & 0 \\ 1 & 2 & 1 & 0 & 3 \end{bmatrix} \quad W = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \\ -1 & 1 & 0 \end{bmatrix}$$

داده شده‌اند. در این مسئله، کانونولوشن با گام (Stride) برابر ۲ و پدینگ (Padding) برابر ۱ انجام می شود.
(آ) خروجی کانونولشن $Y = X * W$ را محاسبه کنید. ابعاد خروجی را مشخص کرده و مقدار هر درایه را بنویسید.
(ب) اگر تابع هزینه به صورت

$$L = \frac{1}{2}\sum_{i,j}(Y_{ij} - T_{ij})^2$$

تعریف شود، مشتقات زیر را استخراج کنید:

$$\frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial W}, \quad \frac{\partial L}{\partial X}.$$

نتیجه را به صورت ماتریسی بنویسید و توضیح دهید که پدینگ و گام چگونه بر ابعاد و مقادیر مشتقات اثر می گذارند.

پاسخ. ◁

(آ) محاسبه خروجی کانولوشن (Forward)

با داشتن

$$X = \begin{bmatrix} 1 & 2 & 0 & 3 & 1 \\ 2 & 1 & 3 & 0 & 2 \\ 0 & 1 & 2 & 1 & 1 \\ 3 & 0 & 1 & 2 & 0 \\ 1 & 2 & 1 & 0 & 3 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \\ -1 & 1 & 0 \end{bmatrix},$$

با گام $s = 2$ و پدینگ $p = 1$ ابتدا X را با یک لایه صفر احاطه می‌کنیم تا X_{pad} به اندازه 7×7 شود. ابعاد خروجی برابر است با

$$H_{\text{out}} = \left\lfloor \frac{7-3}{2} \right\rfloor + 1 = 3, \quad W_{\text{out}} = 3,$$

پس Y ماتریسی 3×3 است.
محاسبه هر درایه با ضرب عنصر به عنصر کرنل در پچ 3×3 متناظر و جمع مؤلفه‌ها انجام می‌شود. نتیجه محاسبات:

$$Y = \begin{bmatrix} 6 & 5 & 4 \\ 3 & 7 & 0 \\ 4 & 0 & 8 \end{bmatrix}$$

(ب) محاسبه مشتقات (Backpropagation)

تابع هزینه را داریم:

$$L = \frac{1}{2}\sum_{i,j}(Y_{ij} - T_{ij})^2.$$

تعریف می‌کنیم:

$$E \equiv \frac{\partial L}{\partial Y} = Y - T,$$

یعنی ماتریس خطا E به صورت

$$E = \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix}, \quad e_{ij} = Y_{ij} - T_{ij}.$$

$$\left[\frac{\partial L}{\partial Y}\right] = E = Y - T$$

(به ازای هر مؤلفه: $\partial L/\partial Y_{ij} = Y_{ij} - T_{ij}$).
$\frac{\partial L}{\partial W}$: برای کانولوشن مورد نظر داریم:

$$Y_{ij} = \sum_{u=0}^{2}\sum_{v=0}^{2} X_{\text{pad}}[r+u, c+v]W[u,v],$$

که $r = i \cdot 2,\ c = j \cdot 2$. بنابراین مشتق نسبت به کرنل حاصل جمع پی‌های ورودی متناظر وزن‌دار با خطاهاست:

$$\frac{\partial L}{\partial W} = \sum_{i=1}^{3}\sum_{j=1}^{3} e_{ij}P_{ij}$$

که P_{ij} پچ 3×3 از X_{pad} است که در تولید Y_{ij} استفاده شده. پچ های عددی به‌شرح زیرند

$$P_{11} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 1 \end{bmatrix}, \quad P_{12} = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 3 \\ 1 & 3 & 0 \end{bmatrix}, \quad P_{13} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 1 \\ 3 & 0 & 2 \end{bmatrix},$$
$$P_{21} = \begin{bmatrix} 0 & 2 & 1 \\ 0 & 0 & 1 \\ 0 & 3 & 0 \end{bmatrix}, \quad P_{22} = \begin{bmatrix} 1 & 3 & 0 \\ 1 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad P_{23} = \begin{bmatrix} 3 & 0 & 2 \\ 2 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix},$$
$$P_{31} = \begin{bmatrix} 0 & 3 & 0 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix}, \quad P_{32} = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad P_{33} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

با جاگذاری مقادیر $e_{ij} = Y_{ij} - T_{ij}$ در رابطه بالا، $\partial L/\partial W$ به‌صورت یک ماتریس 3×3 عددی به‌دست می‌آید.
$\frac{\partial L}{\partial X}$: برای به‌دست‌آوردن گرادیان نسبت به X (ماتریس اصلی بدون پدینگ) ابتدا گرادیان نسبت به X_{pad} را محاسبه می‌کنیم و سپس نواحی پد شده را حذف می‌کنیم. هر اسکالر e_{ij} را با کرنل W در محل متناظر پچ ورودی پخش می‌کنیم. بنابراین

$$\frac{\partial L}{\partial X} = \sum_{i=1}^{3}\sum_{j=1}^{3} e_{ij}S_{ij},$$

که هر S_{ij} ماتریس 5×5 ای است که سهم e_{ij} را روی X نشان می‌دهد (پس از حذف پد). ماتریس‌های S_{ij} به‌صورت عددی عبارت‌اند از:

$$S_{11} = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad S_{12} = \begin{bmatrix} 0 & 0 & 2 & 1 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad S_{13} = \begin{bmatrix} 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$S_{21}=\begin{bmatrix}0&0&0&0&0\\2&1&0&0&0\\1&0&0&0&0\\0&0&0&0&0\\0&0&0&0&0\end{bmatrix},\quad S_{22}=\begin{bmatrix}0&0&0&0&0\\0&0&2&1&0\\0&-1&1&0&0\\0&0&0&0&0\\0&0&0&0&0\end{bmatrix},\quad S_{23}=\begin{bmatrix}0&0&0&0&0\\0&0&0&0&2\\0&0&0&-1&1\\0&0&0&0&0\\0&0&0&0&0\end{bmatrix},$$

$$S_{31}=\begin{bmatrix}0&0&0&0&0\\0&0&0&0&0\\2&1&0&0&0\\1&0&0&0&0\\0&0&0&0&0\end{bmatrix},\quad S_{32}=\begin{bmatrix}0&0&0&0&0\\0&0&0&0&0\\0&0&2&1&0\\0&-1&1&0&0\\0&0&0&0&0\end{bmatrix},\quad S_{33}=\begin{bmatrix}0&0&0&0&0\\0&0&0&0&0\\0&0&0&0&2\\0&0&0&-1&1\\0&0&0&0&0\end{bmatrix}.$$

با ضرب هر S_{ij} در اسکالر خطای متناظر e_{ij} و جمع کردن همهٔ آن‌ها، ماتریس $\frac{\partial L}{\partial X}$ (ابعاد 5×5) به دست می‌آید.

- **پدینگ** باعث می‌شود پچ‌های ورودیِ مربوط به مرزها شامل صفرهای اضافی شوند. بنابراین هم مقدار Y در مرزها و هم مقادیر مشتقات متأثر می‌شوند. برای $\frac{\partial L}{\partial X}$ ابتدا گرادیان را برای X_{pad} محاسبه می‌کنیم و سپس نواحی پد شده را حذف می‌کنیم تا گرادیان نسبت به X اصلی به دست آید.
- **گام (stride)** >1 باعث می‌شود نقاط خروجی از ورودی نمونه‌برداری شده و فاصله‌دار شوند. بنابراین:
  - تعداد موقعیت‌های خروجی کمتر می‌شود (اینجا 3×3 به جای 5×5).
  - هر خطای e_{ij} تنها روی یک پچ مشخص در ورودی تأثیر می‌گذارد (پچ‌هایی که شروعشان با گام فاصله‌گذاری شده‌اند)، و به همین خاطر ماتریس‌های S_{ij} و P_{ij} جابجا شده و پراکنده‌اند.
  - $\frac{\partial L}{\partial W}$ همچنان جمع پچ‌های ورودی وزن‌دار با خطا است، ولی فقط آن پچ‌هایی که مطابق نمونه‌برداری stride انتخاب شده‌اند در جمع نقش دارند.

پرسش ۳. Efficient CNNs (۲۰ نمره)

در سال‌های اخیر، روند طراحی شبکه‌های عصبی کانولوشنی (CNN) به سمت مدل‌های عمیق‌تر و بزرگ‌تر برای دستیابی به دقت بالاتر حرکت کرده است. با این حال، در بسیاری از کاربردهای عملی مانند تلفن‌های هوشمند، منابع محاسباتی بسیار محدود بوده و بودجه محاسباتی معمولا در حد ده‌ها تا چند صد مگافلاپس (MFLOPs) است. ازاین‌رو، تلاش بسیاری برای طراحی معماری‌های کارآمد انجام شده است که بتوانند در چنین محدودیت‌هایی عملکرد مناسبی ارائه دهند. در این سوال به بررسی ایده‌های معماری ShuffleNet می‌پردازیم که منجر به عملکرد مناسب به ازای بودجه محاسباتی محدود می‌شود. این معماری با کاهش هزینه محاسباتی اجزای اصلی شبکه، موفق به ارائه شبکه‌های بزرگ اما کارآمد شده است.

۱ کانولوشن عمقی قابل تفکیک

آ) هزینه محاسباتی کانولوشن استاندارد برای اندازه ورودی $H \times W$، تعداد کانال ورودی C_{in}، تعداد کانال خروجی C_{out} و اندازه فیلتر K را بر حسب FLOPs محاسبه کنید. فرض کنید طول گام برابر یک است و با استفاده از Padding ابعاد مکانی حفظ می‌شود.

ب) کانولوشن عمقی قابل تفکیک [۳] روشی برای کاهش هزینه محاسباتی کانولوشن استاندارد می‌باشد. این روش را به طور کامل توضیح دهید.

ج) مشابه بخش الف، هزینه محاسباتی کانولوشن عمقی قابل تفکیک را بدست آورید و با کانولوشن استاندارد مقایسه کنید.

د) بررسی‌های عملی نشان می‌دهند کانولوشن عمقی قابل تفکیک جایگزین مناسبی برای کانولوشن است. فرض کنید ماتریس کانولوشن به فرم $W \in \mathbb{R}^{C_{out} \times k^2 C_{in}}$ مورد استفاده قرار می‌گیرد. بنابراین برای هر قطعه[۴] $k \times k$ که به فرم $\mathbf{x} \in \mathbb{R}^{k^2 C_{in}}$ تبدیل می‌شود می‌توان حاصل عملیات لایه کانولوشنی را به صورت ضرب ماتریس نمایش داد. ثابت کنید

(آ) کانولوشن عمقی قابل تفکیک ماتریس کانولوشن W را به شکل ضرب دو ماتریس نمایش می‌دهد.

(ب) این ضرب ماتریسی به ازای ماتریس دلخواه W دقیق نخواهد بود.

(ج) با فرض برابر بودن رتبه[۵] ضرب این دو ماتریس با r، این ضرب ماتریسی در حالت کلی بهترین تقریب رنک r از ماتریس W در نرم طیفی [۶] نخواهد بود.

۲ کانولوشن گروهی

در بسیاری از شبکه‌های کانولوشنی، بخش قابل توجهی از هزینه محاسباتی شبکه مربوط به کانولوشن‌های 1×1 [۷] است. همچنین در بخش قبل کاربرد این نوع کانولوشن را دیدیم.

آ) توضیح دهید که کانولوشن 1×1 به چه منظور در معماری‌هایی مانند GoogLeNet و ResNet استفاده می‌شود.

ب) معماری ShuffleNet برای کاهش سربار محاسباتی کانولوشن 1×1 از ایده‌ی کانولوشن گروهی [۸] استفاده می‌کند. این روش را توضیح دهید.

---

[۳] Depthwise Separable Convolution
[۴] Patch
[۵] Rank
[۶] Spectral Norm
[۷] Pointwise Convolution
[۸] Group Convolution

ج) هزینه محاسباتی کانولوشن گروهی را برای اندازه ورودی $H \times W$، تعداد کانال ورودی C_{in}، تعداد کانال خروجی C_{out}، تعداد گروه g و اندازه فیلتر K را بر حسب FLOPs محاسبه کنید. فرض کنید طول گام برابر یک است و با استفاده از padding ابعاد مکانی حفظ می‌شود. آیا این روش منجر به بهبود هزینه محاسباتی در کانولوشن 1×1 می‌شود؟

۳ شافل کردن کانال‌ها

آ) فرض کنید تعدادی لایه با کانولوشن گروهی پشت هم قرار بگیرند. در این صورت چه مشکلی برای جریان اطلاعات و استخراج ویژگی می‌تواند ایجاد شود.

ب) معماری ShuffleNet از ایده شافل کردن کانال‌ها بعد از اعمال کانولوشن گروهی بهره می‌برد. این ایده و نحوه حل کردن مشکل بخش الف را توضیح دهید. همچنین توضیح دهید به چه شکل این روش مشکلی برای جریان گرادیان در شبکه ایجاد نمی‌کند.

پاسخ. ◁

کانولوشن عمقی قابل تفکیک

آ) عملیات کانولوشن از اعمال جمع و ضرب تشکیل شده است که هر کدام یک FLOPs هزینه محاسباتی دارند. هر المان فیچرمپ از یک عملیات کانولوشن ایجاد شده است. چون ابعاد مکانی ورودی حفظ می‌شود $H \times W \times C_{out}$ عملیات کانولوشن انجام می‌شود. هر عملیات کانولوشن روی یک حجم با ابعاد (k, k, C_{in}) انجام می‌شود بنابراین تعداد FLOPs برابر است با:

$$HWC_{out}(2K^2C_{in} - 1 + 1) = 2HWC_{out}K^2C_{in}$$

که یک واحد به خاطر کمتر بودن تعداد عملیات جمع از ضرب کم شده است و یک واحد بخاطر اضافه کردن بایاس بعد هر بار کانوالو کردن اضافه شده است.

ب) در عملیات کانولوشن استاندارد دو عملیات فیلترینگ مکانی و ترکیب کانال‌ها به صورت همزمان انجام می‌شود:

$$(K * I)(i, j, k) = \sum_{m,n,l} K(m, n, l, k)\, I(i + m - 1, j + n - 1, l).$$

کانولوشن عمقی قابل تفکیک این عملیات را در دو مرحله انجام می‌دهد:

(آ) Depthwise Convolution: در این مرحله، هر کانال با فیلتر $k \times k$ خود پردازش می‌شود. بنابراین در این مرحله اطلاعات مکانی هر کانال به طور جداگانه پردازش می‌شود.

$$Y(i, j, k) = \sum_{m,n} K_{Depthwise}(m, n, k)\, I(i + m - 1, j + n - 1, k).$$

(ب) Pointwise Convolution: در مرحله قبل C_{in} فیچرمپ بدست آمد. در این مرحله C_{out} کانولوشن 1×1 خواهیم داشت تا اطلاعات این فیچرمپ‌ها در راستای کانال با یکدیگر ادغام شود و فیچرهای جدید ایجاد شود.

$$Z(i, j, k) = \sum_{l} K_{Pointwise}(l, k)\, Y(i, j, l)$$

بنابراین کانولوشن عمقی قابل تفکیک فرض می‌کند که کورلیشن مکانی مستقل از کورلیشن بین کانال‌ها است.

ج) هزینه محاسباتی Depthwise convolution برابر است با

$$HWC_{in}(2K^2 - 1 + 1) = 2HWC_{in}K^2$$

و هزینه محاسباتی Pointwise convolution برابر است با

$$HWC_{out}(2C_{in} - 1 + 1) = 2HWC_{out}C_{in}$$

در بسیاری از تنظیمات این هایپرپارامترها، $2HWK^2C_{in}C_{out}$ عبارت غالب خواهد بود و بنابراین کانولوشن سربار محاسباتی بیشتری خواهد داشت.

$$\frac{FLOPs_{Depthwisesep.conv.}}{FLOPs_{conv.}} = \frac{2HWC_{in}(K^2 + C_{out})}{2HWC_{in}C_{out}K^2} = \frac{1}{C_{out}} + \frac{1}{k^2}$$

د) (آ) با توجه به بخش ب، می‌توان ماتریس کانولوشن را به شکل زیر تقریب زد:

$$\underbrace{\hat{W}}_{C_{\text{out}} \times (k^2 C_{\text{in}})} = \underbrace{P}_{C_{\text{out}} \times C_{\text{in}}} \underbrace{D}_{C_{\text{in}} \times (k^2 C_{\text{in}})} = P \begin{bmatrix} d_1 & 0 & \dots & 0 \\ 0 & d_2 & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & d_{C_{\text{in}}} \end{bmatrix}, \quad d_i \in \mathbb{R}^{1 \times k^2}.$$

در این factorization، هر ماتریس d_i متناظر با عملیات Depthwise convolution در کانال iام است و هر سطر ماتریس P متناظر با یک Pointwise convolution است. توجه شود فرض می‌کنیم ماتریس W در flat شده هر پچ ضرب می‌شود.

(ب) با ضرب ماتریس P در ماتریس بلوکی D خواهیم داشت:

$$\hat{W} = [\, p_1 d_1 \; p_2 d_2 \; \dots \; p_{c_{\text{in}}} d_{c_{\text{in}}} \,]$$

ماتریس W را به فرم بلوکی ماتریس $\hat{W}$ در می‌آوریم:

$$W = [\, W_1 \; W_2 \; \dots \; W_{C_{in}} \,], \quad W_i \in \mathbb{R}^{C_{out} \times k^2}$$

بنابراین هر ماتریس W_i به ضرب خارجی بردارهای p_i و d_i تقریب زده شده است. توجه داریم رنک ماتریس حاصل از ضرب خارجی دو بردار همواره برابر یک است. از طرفی $Rank(W_i) \leq min(C_{out}, k^2)$ است و لزوما رنک یک نمی‌باشد. بنابراین می‌بینیم هر ماتریس W_i با یک تقریب رنک یک در کانولوشن عمقی قابل تفکیک تقریب زده می‌شود و چون رنک ماتریس W_i لزوما یک نمی‌باشد، بنابراین کانولوشن عمقی قابل تفکیک به طور کلی دقیق نمی‌باشد.

(ج) با استفاده از SVD می‌توان ماتریس W را به شکل $W = U\Sigma V^T$ نوشت. طبق قضیه Eckart–Young–Mirsky، بهترین تقریب مرتبه r از این ماتریس به فرم $W_r = U_r \Sigma_r V_r^T$ است که در آن ماتریس‌های V_r و U_r شامل r ستون اول ماتریس‌های V و U می‌باشند و ماتریس قطری Σ_r دارای r مقدار تکین بزرگ است. بنابراین داریم:

$$U_r \Sigma_r V_r^T = \arg \min_{\text{rank}(A) \leq r} \|W - A\|_2,$$

$$\min_{\text{rank}(A) \leq r} \|W - A\|_2 \; \leq \; \|W - PD\|_2 \,.$$

توجه شود ماتریس D یک ماتریس قطری بلوکی است. این در حالی است که $\Sigma_r V_r^T$ به طور کلی به این فرم نمی‌باشد. بنابراین، کانولوشن عمقی قابل تفکیک در فضای محدودتری جست و جو می‌کند و لزوما به تقریب مرتبه پایین بهینه که همان truncated SVD است نمی‌رسد.

کانولوشن گروهی

آ) از این کانولوشن برای تغییر ابعاد در راستای کانال استفاده می‌شود. بنابراین هم می‌توان به آن به چشم یک $LinearProjection$ نگاه کرد که بردارهای فیچر را به یک فضای با ابعاد پایین‌تر می‌برد و هم می‌توان از آن برای افزایش تعداد کانال استفاده کرد. در معماری GoogLeNet از این اپراتور در Inception units استفاده می‌شود. هدف ترکیب و کاهش ابعاد فیچرها (کانال‌ها) است تا هزینه محاسباتی و تعداد پارامترها کاهش یابد. در معماری ResNet نیز در دو مرحله از آن در گلوگاه استفاده می‌شود. بار نخست برای فشرده‌سازی فیچرها برای کاهش محاسبات و بار دوم برای افزایش تعداد کانال‌ها و سازگار شدن آن با خروجی shortcut استفاده می‌شود. با استفاده از این ساختار می‌توان ضمن کنترل محاسبات و تعداد پارامترها، یک گلوگاه عمیق‌تر داشت.

به طور کلی کانولوشن 1×1 متناظر با یک نورون خطی است و اعمال آن معادل با ضرب یک ماتریس در $H \times W$ بردار با تعداد C_{in} درایه است.

ب) در کانولوشن گروهی، کانال‌های ورودی و فیلترها به g گروه تقسیم می‌شوند و هر گروه با فیلترهای همان گروه کانوالو می‌شود. در واقع هر فیلتر اکنون $\frac{C_{in}}{g}$ کانال دارد و تعداد خروجی‌های هر گروه برابر $\frac{C_{out}}{g}$ فیچرمپ است که تنها به g کانال از ورودی دسترسی دارند.

ج) هزینه محاسباتی کانولوشن گروهی برابر است با

$$HWC_{out}(\frac{2K^2C_{in}}{g} - 1 + 1) = \frac{2}{g}HWK^2C_{in}C_{out}$$

و بنابراین هزینه محاسباتی آن به اندازه تعداد گروه اسکیل می‌شود و به ازای $g > 1$ هزینه محاسباتی کمتری خواهد داشت.

شافل کردن کانال‌ها

آ) در این صورت هر کانال خروجی بعد از اعمال چند لایه تنها به تعداد کمی از کانال‌های ورودی وابسته می‌شود. این موضوع باعث می‌شود که جریان اطلاعات به خوبی در طول شبکه منتقل نشود و قدرت بازنمایی شبکه تضعیف می‌شود.

به عنوان مثال فرض کنید دو لایه کانولوشن گروهی با تعداد کانال و گروه یکسان پشت یکدیگر قرار بگیرند. در این صورت گروه iام در لایه دوم تنها به همین گروه در لایه پیشین دسترسی دارد. این موضوع باعث می‌شود که شبکه به چند شبکه مستقل و کوچک تبدیل شود که دسترسی محدودتری به فیچرها دارد. از طرفی این استقلال باعث می‌شود که جریان گرادیان هر گروه مستقل از گروه دیگر باشد. بنابراین امکان ساخت فیچرهای ترکیبی و پیشرفته در لایه‌های آخر شبکه توسط فیچرهای ساده لایه‌های ابتدایی تا حد زیادی از بین می‌رود چون یک گراف ارتباط بسیار تنک در شبکه‌های این چنینی ایجاد می‌شود.

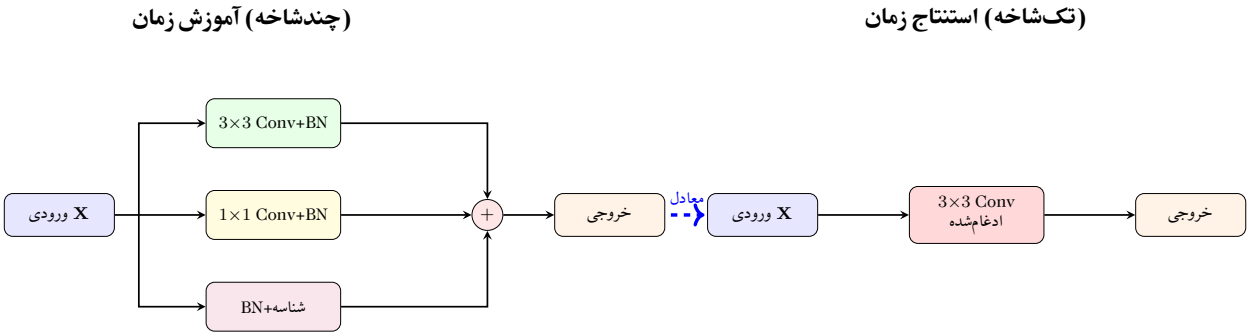
ب) برای حل مشکل بخش الف، از ایده جابجا کردن کانال‌ها بین گروه‌های مختلف استفاده می‌شود. در واقع بعد هر لایه کانولوشن گروهی، یک عملیات شافل کردن انجام می‌شود. با انجام این کار مسیرهای جدیدی برای جریان اطلاعات ایجاد می‌شود و فیچرهای متنوع‌تری در طول شبکه یاد گرفته می‌شود. در واقع عملیات شافل کردن باعث می‌شود که فیچرهای پیشرفته موجود در لایه‌های عمیق، از گروه‌های مختلف ایجاد شود. توجه شود که عملیات شافل کردن به تدریج امکان استفاده از اطلاعات تمام کانال‌ها را فراهم می‌کند. همچنین به دلیل وابسته شدن گروه‌های مختلف، گرادیان یک گروه بر چند گروه در لایه قبلی اثر می‌گذارد که منجر به فرآیند یادگیری یکپارچه می‌شود و از ایجاد زیرشبکه‌های مستقل جلوگیری می‌کند.

نحوه پیاده‌سازی این عملیات به این شکل است که ابتدا تنسور فیچرمپ به یک تنسور با ابعاد $(H, W, g, C/g)$ بازآرایی می‌شود. سپس نسبت به دو بعد انتهایی ترنسپوز می‌شود و دوباره flat می‌شود. با اینکار ترتیب کانال‌ها دیگر براساس گروه‌ها نیست و در صورت گروه‌بندی، از گروه‌های مختلف کانال وجود خواهد داشت.

همچنین این عملیات کاملا مشتق‌پذیر است چون معادل با ضرب ماتریس permutation است.

پرسش ۴. بازپارامتری‌سازی ساختاری: از چندشاخه به تک‌شاخه [۲۵ نمره]

پیش‌زمینه: معماری‌هایی مثل ResNet و GoogLeNet از ساختارهای **چندشاخه‌ای** برای بهبود جریان گرادیان استفاده می‌کنند. اما در زمان استنتاج (Inference)، هر شاخه‌ی اضافی نیاز به حافظه و محاسبات مستقل دارد. در سال ۲۰۲۱، ایده‌ی **بازپارامتری‌سازی ساختاری** (Structural Re-parameterization) در معماری RepVGG مطرح شد: در زمان آموزش از چند شاخه استفاده کنیم و در زمان استنتاج، همه‌ی شاخه‌ها را در یک کانولوشن واحد ادغام (Fuse) کنیم. مزیت: هم از مزایای آموزش چندشاخه‌ای بهره می‌بریم و هم سرعت استنتاج تک‌شاخه‌ای را داریم.



شکل ۱: بازپارامتری‌سازی ساختاری: بلوک چندشاخه‌ای آموزش (چپ) به بلوک تک‌شاخه‌ای استنتاج (راست) تبدیل می‌شود. همه‌ی شاخه‌ها در یک کانولوشن 3×3 ادغام می‌شوند.

بخش اول: ادغام کانولوشن و نرمال‌سازی دسته‌ای

برای ورودی $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ و کرنل $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}$، کانولوشن تعریف می‌شود:

$$(\mathbf{W} * \mathbf{X})_{c,i,j} = \sum_{c'=1}^{C_{\text{in}}} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} W_{c,c',p,q} \cdot X_{c',i+p,j+q}$$

نرمال‌سازی دسته‌ای (BN) برای بردار ویژگی $\mathbf{y} \in \mathbb{R}^C$:

$$\text{BN}(\mathbf{y}; \gamma, \beta, \mu, \sigma^2) = \gamma \odot \frac{\mathbf{y} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

که $\gamma, \beta \in \mathbb{R}^C$ پارامترهای آموخته‌شده، $\mu, \sigma^2 \in \mathbb{R}^C$ آمارهای اجرایی، و $\epsilon > 0$ ثابت پایداری است.

آ) ادغام کانولوشن و BN. نشان دهید که ترکیب:

$$y_c = \gamma_c \cdot \frac{(\mathbf{W}_c * \mathbf{X} + b_c - \mu_c)}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c$$

می‌تواند به صورت یک کانولوشن معادل نوشته شود. **فرمول‌های دقیق** وزن معادل $\mathbf{W}'_c$ و بایاس معادل b'_c را بر حسب $\mathbf{W}_c, b_c, \gamma_c, \beta_c, \mu_c, \sigma_c^2, \epsilon$ استخراج کنید.

ب) تبدیل کانولوشن 1×1 به 3×3. یک کانولوشن 1×1 با وزن $\mathbf{W}_{1\times1} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 1 \times 1}$ داده شده. کرنل معادل 3×3 یعنی $\tilde{\mathbf{W}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 3 \times 3}$ را تعریف کنید. دقیقاً مشخص کنید کدام درایه‌های $\tilde{\mathbf{W}}$ ناصفر هستند و مقادیرشان چیست.

ج) قضیه‌ی ادغام شاخه‌های موازی. ثابت کنید که اگر $\mathbf{W}^{(1)}$ و $\mathbf{W}^{(2)}$ دو کانولوشن 3×3 به صورت موازی اعمال شوند (ورودی یکسان، خروجی‌ها جمع می‌شوند):

$$\mathbf{W}^{(\text{fused})} * \mathbf{X} = \mathbf{W}^{(1)} * \mathbf{X} + \mathbf{W}^{(2)} * \mathbf{X}$$

فرم دقیق $\mathbf{W}^{(\text{fused})}$ چیست؟

بخش دوم: نگاشت شناسه به عنوان کانولوشن و ادغام بلوک ResNet

در بخش اول دیدیم که هر شاخه‌ی موازی (از جمله شاخه‌ی شناسه) را می‌توان در یک کانولوشن 3×3 ادغام کرد. اما آیا نگاشت شناسه $\mathrm{id}(\mathbf{X}) = \mathbf{X}$ همیشه به صورت یک کانولوشن نمایش‌پذیر است؟ جواب منوط به ابعاد ورودی و خروجی است. برای $C_{\text{in}} = C_{\text{out}} = C$ و $\text{stride} = 1$، نگاشت شناسه می‌تواند به صورت یک کانولوشن 3×3 نمایش داده شود (کرنل شناسه). اما اگر این شرط‌ها برقرار نباشند، وضعیت متفاوت می‌شود.

آ) کرنل شناسه $\mathbf{W}_{\text{id}} \in \mathbb{R}^{C \times C \times 3 \times 3}$ را **به طور صریح** بنویسید. کدام درایه‌ها ناصفر هستند و مقادیرشان چیست؟ نشان دهید که اعمال این کرنل روی ورودی $\mathbf{X}$ دقیقاً $\mathbf{X}$ را برمی‌گرداند.

ب) حال دو حالت زیر را جداگانه بررسی کنید:

i. اگر $C_{\text{in}} \neq C_{\text{out}}$: نشان دهید که هیچ کرنل $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 3 \times 3}$ وجود ندارد که $\mathbf{W} * \mathbf{X} = \mathbf{X}$ برای همه‌ی $\mathbf{X}$. راهنمایی: ابعاد ورودی و خروجی کانولوشن را بررسی کنید.

ii. اگر $\text{stride} = s > 1$: نشان دهید که خروجی کانولوشن با $s\text{stride}$ = ابعاد مکانی $\frac{H}{s} \times \frac{W}{s}$ دارد و بنابراین نمی‌تواند برابر $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ باشد.

ResNet در این حالت‌ها از **پروجکشن میان‌بر** (Projection Shortcut) استفاده می‌کند: یک کانولوشن 1×1 با stride مناسب که ابعاد را تطبیق می‌دهد. توضیح دهید که چرا این کانولوشن 1×1 هم‌ارز نگاشت شناسه نیست و چه اطلاعاتی احتمالاً از دست می‌روند.

ج) برای یک بلوک ResNet با شاخه اصلی $\mathbf{W}_{\text{main}}$ (شامل دو کانولوشن 3×3 متوالی هر کدام با BN) و اتصال میان‌بر شناسه، فرمول کرنل ادغام‌شده $\mathbf{W}_{\text{fused}}$ را پس از بازپارامتری‌سازی بنویسید. راهنمایی: ابتدا هر کانولوشن+BN را ادغام کنید، سپس شاخه‌های موازی را با هم جمع کنید.

بخش سوم: تحلیل نظری

آ) **قضیه‌ی هم‌ارزی.** ثابت کنید که قدرت بازنمایی مدل ادغام‌شده **دقیقاً هم‌ارز** مدل چندشاخه‌ای است. یعنی برای هر ورودی $\mathbf{X}$:

$$\mathcal{F}_{\text{multi}}(\mathbf{X}) = \mathcal{F}_{\text{fused}}(\mathbf{X})$$

چه فرض‌هایی برای این هم‌ارزی لازم است؟

ب) **تحلیل FLOPs.** برای یک بلوک ResNet با:

- شاخه اصلی: $C \to C$ با کرنل 3×3
- اتصال میان‌بر: $C \to C$ با کرنل 1×1 (پروجکشن)

FLOPs کل را قبل و بعد از ادغام برای ابعاد مکانی $H = W = 56$ و $C = 256$ محاسبه کنید. نشان دهید FLOPs تغییر میکند.

ج) **تحلیل دسترسی حافظه.** نشان دهید که هزینه دسترسی حافظه برای مدل چندشاخه‌ای تقریباً برابر است با:

$$\text{MAC}_{\text{multi}} \approx 2 \cdot C^2 \cdot H \cdot W + 2 \cdot C \cdot H \cdot W$$

در حالی که برای مدل ادغام‌شده:

$$\text{MAC}_{\text{fused}} \approx C^2 \cdot H \cdot W + 2 \cdot C \cdot H \cdot W$$

چرا این تفاوت برای کارایی سخت‌افزاری مهم است؟ مفهوم **شدت حسابی** (Arithmetic Intensity) را توضیح دهید و نشان دهید چرا مدل ادغام‌شده از نظر سخت‌افزاری کارآمدتر است.

پاسخ. بخش اول: ادغام کانولوشن و BN

آ) ترکیب کانولوشن و BN:

$$y_c = \gamma_c \cdot \frac{(\mathbf{W}_c * \mathbf{X} + b_c - \mu_c)}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c = \underbrace{\frac{\gamma_c}{\sqrt{\sigma_c^2 + \epsilon}}}_{\alpha_c} \cdot (\mathbf{W}_c * \mathbf{X}) + \underbrace{\frac{\gamma_c(b_c - \mu_c)}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c}_{\beta'_c}$$

پس وزن معادل و بایاس معادل:

$$\mathbf{W}'_c = \frac{\gamma_c}{\sqrt{\sigma_c^2 + \epsilon}} \cdot \mathbf{W}_c, \quad b'_c = \frac{\gamma_c(b_c - \mu_c)}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c$$

یعنی هر کرنل کانولوشن به صورت یکنواخت مقیاس می‌شود ($\alpha_c = \gamma_c / \sqrt{\sigma_c^2 + \epsilon}$ ضرب در همه‌ی وزن‌ها) و بایاس جدید ترکیبی از بایاس قبلی، میانگین BN و پارامتر شیفت است.

ب) تبدیل کانولوشن 1×1 به 3×3: کرنل $\tilde{\mathbf{W}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 3 \times 3}$ به صورت زیر تعریف می‌شود:

$$\tilde{W}_{c,c',p,q} = \begin{cases} W^{1\times1}_{c,c',0,0} & \text{if} p = 1 \text{ and } q = 1 \\ 0 & \text{otherwise} \end{cases}$$

یعنی فقط درایه‌ی مرکزی ($p = 1, q = 1$) از هر فیلتر 3×3 ناصفر است و مقدارش برابر درایه‌ی متناظر در کرنل 1×1. این در واقع کرنل 1×1 است که با صفر پدینگ شده تا ابعاد 3×3 پیدا کند.

ج) ادغام دو کانولوشن موازی: چون خطی بودن کانولوشن:

$$(\mathbf{W}^{(1)} + \mathbf{W}^{(2)}) * \mathbf{X} = \mathbf{W}^{(1)} * \mathbf{X} + \mathbf{W}^{(2)} * \mathbf{X}$$

پس $\mathbf{W}^{(\text{fused})} = \mathbf{W}^{(1)} + \mathbf{W}^{(2)}$ و $b^{(\text{fused})} = b^{(1)} + b^{(2)}$. به عبارت ساده، کرنل‌ها و بایاس‌ها جمع می‌شوند. این به لطف خطی بودن عملیات کانولوشن امکان‌پذیر است.

▷

پاسخ. بخش دوم: نگاشت شناسه و ادغام بلوک ResNet

آ) کرنل شناسه $\mathbf{W}_{\text{id}} \in \mathbb{R}^{C \times C \times 3 \times 3}$:

$$W_{\text{id},c,c',p,q} = \begin{cases} 1 & \text{if} c = c' \text{ and } p = 1 \text{ and } q = 1 \\ 0 & \text{otherwise} \end{cases}$$

یعنی فقط درایه‌ی مرکزی ($p = 1, q = 1$) از هر بلوک کانالی $c = c'$ برابر ۱ و بقیه صفر. این عملاً یک کانولوشن 1×1 با ماتریس وزن $\mathbf{I}$ است که در مرکز کرنل 3×3 قرار گرفته. برای نشان دادن اینکه این کرنل $\mathbf{X}$ را برمی‌گرداند:

$$(\mathbf{W}_{\text{id}} * \mathbf{X})_{c,i,j} = \sum_{c'=1}^{C} \sum_{p=0}^{2} \sum_{q=0}^{2} W_{\text{id},c,c',p,q} \cdot X_{c',i+p,j+q} = \sum_{c'=1}^{C} \delta_{c,c'} \cdot X_{c',i+1,j+1} = X_{c,i,j}$$

(دقت کنید که اندیس‌های $i+1, j+1$ به دلیل موقعیت مرکزی $p = 1, q = 1$ و padding = 1 به $X_{c,i,j}$ تبدیل می‌شوند.)

ب) دو حالت را جداگانه بررسی می‌کنیم:

i. اگر $C_{\text{in}} \neq C_{\text{out}}$: نگاشت شناسه ورودی $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ را به همان $\mathbf{X}$ نگاشت می‌دهد، اما خروجی باید C_{out} کانال داشته باشد. یک کانولوشن استاندارد وزن‌های $\mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}$ دارد. برای $\mathbf{W} * \mathbf{X} = \mathbf{X}$، باید $C_{\text{out}} = C_{\text{in}}$ باشد زیرا خروجی کانولوشن C_{out} کانال دارد اما $\mathbf{X}$ فقط C_{in} کانال دارد. اگر $C_{\text{out}} > C_{\text{in}}$، کانولوشن نمی‌تواند ماتریس همانی غیرمربعی را بازتولید کند (زیرا کانال‌های اضافی خروجی باید صفر باشند، نه برابر ورودی).

ii. اگر stride $:= s > 1$ خروجی کانولوشن با $s$stride = ابعاد مکانی $\lfloor \frac{W}{s} \rfloor \times \lfloor \frac{H}{s} \rfloor$ دارد، اما $\mathbf{X}$ ابعاد $H \times W$ دارد. چون $\lfloor \frac{H}{s} \rfloor < H$، خروجی نمی‌تواند برابر $\mathbf{X}$ باشد.

ResNet در این حالت‌ها از «پروجکشن شورت‌کات» استفاده می‌کند: یک کانولوشن 1×1 با stride مناسب که ابعاد را تطبیق می‌دهد. این کانولوشن هم‌ارز نگاشت شناسه نیست زیرا: (۱) وقتی $C_{\text{in}} \neq C_{\text{out}}$، یک کانولوشن 1×1 خطی ترکیب کانال‌ها را تغییر می‌دهد و نمی‌تواند ماتریس همانی غیرمربعی را بسازد — اطلاعات برخی کانال‌ها ممکن است از دست بروند یا کانال‌های جدید ترکیب خطی کانال‌های قبلی باشند. (۲) وقتی 1stride $>$، اطلاعات مکانی بین نمونه‌برداری‌شده از دست می‌رود.

ج) ادغام بلوک ResNet (بدون غیرخطی بین دو کانولوشن):

i. **ادغام هر کانولوشن و BN** (طبق بخش ۱ـ الف):

$$\mathbf{W}'_1 = \frac{\gamma_1}{\sqrt{\sigma_1^2 + \epsilon}} \mathbf{W}_1, \quad b'_1 = \frac{\gamma_1 (b_1 - \mu_1)}{\sqrt{\sigma_1^2 + \epsilon}} + \beta_1$$

$$\mathbf{W}'_2 = \frac{\gamma_2}{\sqrt{\sigma_2^2 + \epsilon}} \mathbf{W}_2, \quad b'_2 = \frac{\gamma_2 (b_2 - \mu_2)}{\sqrt{\sigma_2^2 + \epsilon}} + \beta_2$$

ii. **ترکیب دو کانولوشن متوالی:** چون بین دو کانولوشن هیچ تابع غیرخطی (مانند ReLU) وجود ندارد، ترکیب آن‌ها معادل یک کانولوشن واحد با کرنل 5×5 است:

$$W_{\text{comp}}[j, c, r, s] = \sum_i \sum_{p'=0}^{2} \sum_{q'=0}^{2} W'_2[j, i, p', q'] \; W'_1[i, c, r - p', s - q']$$

که $r, s \in \{0, 1, 2, 3, 4\}$ و $W'_1[i, c, u, v] = 0$ اگر $u, v \notin \{0, 1, 2\}$. بایاس مؤلفه:

$$b_{\text{comp}}[j] = \sum_i \sum_{p'=0}^{2} \sum_{q'=0}^{2} W'_2[j, i, p', q'] \; b'_1[i] \; + \; b'_2[j]$$

iii. **جمع با میان‌بر شناسه:** کرنل شناسه 3×3 را با صفر پدینگ به 5×5 تبدیل می‌کنیم (دلتا در مرکز $(r = 2, s = 2)$):

$$W_{\text{id}}^{5\times5}[j, c, r, s] = \begin{cases} 1 & c = j, \; r = 2, \; s = 2 \\ 0 & \text{otherwise} \end{cases}$$

iv. **کرنل ادغام‌شده نهایی:**

$$\boxed{\mathbf{W}_{\text{fused}} = \mathbf{W}_{\text{comp}} + \mathbf{W}_{\text{id}}^{5\times5}}$$

$$\boxed{b_{\text{fused}} = b_{\text{comp}}}$$

نکته: کرنل ادغام‌شده 5×5 است، نه 3×3. شکل سوال بلوک 3×3 را نشان می‌دهد که مربوط به بلوک RepVGG با سه شاخه **موازی** است. در اینجا دو کانولوشن **متوالی** هستند و با ترکیب (composition) به 5×5 تبدیل می‌شوند. همچنین در این پاسخ تابع فعال ساز ReLU بین دو دو تابع پیچشی درنظر گرفته نشده اما اگر شما درنظر گرفتید مشکلی ندارد و پاسخ زیر هم صحیح است:

i. هر کانولوشن + BN را ادغام می‌کنیم: $\mathbf{W}'_1, b'_1$ (کانولوشن اول + BN اول) و $\mathbf{W}'_2, b'_2$ (کانولوشن دوم + BN دوم)

ii. شاخه اصلی دو کانولوشن متوالی دارد: خروجی $= \mathbf{W}'_2 * (\mathbf{W}'_1 * \mathbf{X} + b'_1) + b'_2$. این معادل یک کانولوشن واحد نیست (زیرا غیرخطی ReLU بین آن‌هاست).

iii. اما اتصال میان‌بر شناسه + شاخه اصلی را می‌توان ادغام کرد. اگر فرض کنیم غیرخطی ReLU بعد از جمع قرار دارد، آنگاه:

$$\mathbf{W}_{\text{fused}} = \mathbf{W}'_{\text{main}} + \mathbf{W}_{\text{id}}, \quad b_{\text{fused}} = b'_{\text{main}} + 0$$

که $\mathbf{W}'_{\text{main}}$ نتیجه‌ی ادغام دو کانولوشن متوالی + BN هر کدام + ReLU بینشان است. توجه: چون ReLU غیرخطی است، $\mathbf{W}'_{\text{main}}$ نمی‌تواند به سادگی $\mathbf{W}'_2 * \mathbf{W}'_1$ باشد. در عمل، RepVGG فقط بلوک‌هایی با یک کانولوشن در شاخه اصلی + اتصال شناسه + شاخه 1×1 ادغام می‌کند (نه دو کانولوشن متوالی).

◁

پاسخ. بخش سوم: تحلیل نظری

آ) قضیه‌ی هم‌ارزی: برای هر ورودی $\mathbf{X}$، نشان می‌دهیم $\mathcal{F}_{\text{multi}}(\mathbf{X}) = \mathcal{F}_{\text{fused}}(\mathbf{X})$. مراحل اثبات:

i. هر شاخه $(\mathbf{W}^{(k)} * \mathbf{X} + b^{(k)})$ با BN ادغام می‌شود: خروجی $= \mathbf{W}'^{(k)} * \mathbf{X} + b'^{(k)}$. این دقیقاً هم‌ارز است زیرا فرمول BN فقط مقیاس و شیفت خطی اعمال می‌کند.

ii. شاخه‌های موازی جمع می‌شوند: $\sum_k (\mathbf{W}'^{(k)} * \mathbf{X} + b'^{(k)}) = (\sum_k \mathbf{W}'^{(k)}) * \mathbf{X} + \sum_k b'^{(k)}$. این هم‌ارز است زیرا کانولوشن خطی است.

فرض‌های لازم: (۱) شاخه‌ها مستقل هستند (ورودی یکسان، خروجی‌ها جمع می‌شوند)، (۲) BN در حالت ارزیابی است (از آمارهای اجرایی استفاده می‌شود، نه آمار بچ)، (۳) هیچ غیرخطی بین شاخه‌ها و جمع نیست.

ب) تحلیل FLOPs

برای بلوکی با شاخه اصلی 3×3 $(C \to C)$ و اتصال میان‌بر 1×1 $(C \to C)$، پروجکشن) و ابعاد $H = W = 56$، $C = 256$:

$$C = 256, \quad H \cdot W = 3136, \quad C^2 = 65536, \quad C \cdot H \cdot W = 802816$$

قبل از ادغام:

- شاخه اصلی 3×3: $C \cdot C \cdot 9 \cdot H \cdot W = 65536 \times 9 \times 3136 = \mathbf{1\,849\,688\,064}$
- اتصال میان‌بر 1×1: $C \cdot C \cdot 1 \cdot H \cdot W = 65536 \times 3136 = \mathbf{205\,520\,896}$
- جمع دو شاخه ($C \cdot H \cdot W$ عمل جمع): $256 \times 3136 = \mathbf{802\,816}$

$$\text{FLOPs}_{\text{قبل}} = 1\,849\,688\,064 + 205\,520\,896 + 802\,816 = \mathbf{2\,056\,011\,776}$$

پس از ادغام: شاخه 1×1 در درایه مرکزی کرنل 3×3 ادغام شده و عملیات جمع صریح حذف می‌شود. تنها یک کانولوشن 3×3 با $C \to C$ باقی می‌ماند:

$$\text{FLOPs}_{\text{بعد}} = C \cdot C \cdot 9 \cdot H \cdot W = 65536 \times 9 \times 3136 = \mathbf{1\,849\,688\,064}$$

نتیجه: FLOPs پس از ادغام **کاهش می‌یابد**:

$$\Delta = 2\,056\,011\,776 - 1\,849\,688\,064 = \mathbf{206\,323\,712}$$

$$\text{درصد کاهش} \approx \frac{206\,323\,712}{2\,056\,011\,776} \times 100 \approx \mathbf{10.03}\%$$

ج) تحلیل دسترسی حافظه (Memory Access Cost — MAC)

مدل ساده‌شده برای تحلیل: در این تحلیل، با صرف‌نظر از ابعاد فضایی کرنل، تعداد پارامترهای هر شاخه را C^2 در نظر می‌گیریم (چه 3×3 و چه 1×1). همچنین فرض می‌کنیم پارامترها برای **هر پیکسل خروجی** یک بار از حافظه خوانده می‌شوند (بدون کش). هزینهٔ دسترسی به نگاشت‌های ویژگی نیز شامل خواندن ورودی و نوشتن خروجی نهایی است ($2 \cdot C \cdot H \cdot W$).

مدل چندشاخه‌ای (دو شاخه 3×3 و 1×1):

- خواندن وزن دو شاخه: $2 \times C^2 \cdot H \cdot W$
- خواندن ورودی و نوشتن خروجی: $2 \cdot C \cdot H \cdot W$

$$\boxed{\text{MAC}_{\text{multi}} \approx 2C^2HW + 2CHW}$$

مدل ادغام‌شده (یک کانولوشن 3×3):

- خواندن وزن یک شاخه: $C^2 \cdot H \cdot W$
- خواندن ورودی و نوشتن خروجی: $2 \cdot C \cdot H \cdot W$

$$\boxed{\text{MAC}_{\text{fused}} \approx C^2HW + 2CHW}$$

مقایسه عددی (برای $C = 256,\ H = W = 56$):

$$\text{MAC}_{\text{multi}} = 2 \times 65536 \times 3136 + 2 \times 802816 = 411\,041\,792 + 1\,605\,632 = \mathbf{412\,647\,424}$$

$$\text{MAC}_{\text{fused}} = 65536 \times 3136 + 2 \times 802816 = 205\,520\,896 + 1\,605\,632 = \mathbf{207\,126\,528}$$

یعنی MAC مدل ادغام‌شده تقریباً **نصف** مدل چندشاخه‌ای است.

شدت حسابی (Arithmetic Intensity):

$$\text{AI} = \frac{\text{FLOPs}}{\text{MAC}}$$

$$\text{AI}_{\text{multi}} = \frac{10\,C^2HW}{2C^2HW + 2CHW} = \frac{5C}{C+1} \xrightarrow{C \gg 1} 5$$

$$\mathrm{AI}_{\text{fused}} = \frac{9\,C^2HW}{C^2HW + 2CHW} = \frac{9C}{C+2} \xrightarrow{C\gg 1} 9$$

$$\boxed{\frac{\mathrm{AI}_{\text{fused}}}{\mathrm{AI}_{\text{multi}}} \approx \frac{9}{5} = \mathbf{1.8}}$$

تفسیر: مدل ادغام‌شده شدت حسابی 1.8 رابری دارد. در پردازنده‌های گرافیکی، وقتی شدت حسابی پایین باشد، سیستم **محدود به حافظه** (memory-bound) می‌شود و واحدهای محاسباتی بیکار می‌مانند. بازپارامتری‌سازی ساختاری با حذف شاخه‌های موازی و ادغام آن‌ها، دسترسی به حافظهٔ واسط را حذف کرده و نسبت محاسبه به انتقال داده را افزایش می‌دهد. نتیجه: بهره‌وری سخت‌افزاری بالاتر و سرعت استنتاج بیشتر، بدون افت دقت.

▷

پرسش ۵. الگوریتم پس‌انتشار و چالش‌های آن برای RNN (۱۵ نمره)

بخش اول

یک نسخهٔ ساده و تک‌بعدی RNN را به شکل زیر فرض کنید

$$h_t = \tanh(w \cdot h_{t-1}), \quad t \in \mathbb{N}$$

که در آن $h_t \in \mathbb{R}$ نشان‌گر حالت پنهانی شبکه و $w \in \mathbb{R}$ پارامتر قابل یادگیری است. همچنین فرض نمایید این شبکه تنها در گام پایانی Tـام خروجی می‌دهد به طوری که

$$\hat{y} = h_T.$$

برای تابع اتلاف نیز تابع مربعات خطا به‌شکل زیر برای برچسب y^* استفاده می‌شود:

$$L = \frac{1}{2}(h_T - y^*)^2.$$

برای آموزش شبکه‌های RNN از الگوریتم پس‌انتشار طی زمان (BPTT) استفاده می‌شود؛ در این راستا نشان دهید رابطهٔ گرادیان خطا نسبت به پارامتر مدل، به شکل زیر است:

$$\frac{\partial L}{\partial w} = (h_T - y^*) \sum_{t=1}^{T} w^{T-t} \cdot (1 - h_t^2) h_{t-1} \cdot \prod_{j=t+1}^{T} (1 - h_j^2)$$

یادآوری: توجه کنید که رابطهٔ $\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$ برقرار است.

بخش دوم

آ) فرض کنید $\gamma_i := w \cdot (1 - h_i^2)$. بازهٔ مقادیر ممکن برای $|\gamma_i|$ را مشخص نمایید.

ب) گام مشخص t را در نظر بگیرید و طول دنبالهٔ را $T \to \infty$ فرض کنید. مشخص نمایید تحت چه شرایطی چالش محو شدن گرادیان (Gradient Vanishing) گرادیان رخ می‌دهد؟

ج) گام مشخص t را در نظر بگیرید و طول دنبالهٔ را $T \to \infty$ فرض کنید. مشخص نمایید تحت چه شرایطی چالش انفجار (exploding) گرادیان رخ می‌دهد؟

پاسخ. ▷

بخش اول

می‌دانیم تابع هزینه فقط به h_T وابسته است:

$$L = \frac{1}{2}(h_T - y^*)^2 \quad \Rightarrow \quad \frac{\partial L}{\partial w} = (h_T - y^*) \cdot \frac{\partial h_T}{\partial w}.$$

حال مشتق $\frac{\partial h_T}{\partial w}$ را به کمک قاعدهٔ زنجیره‌ای می‌نویسیم:

$$\frac{\partial h_T}{\partial w} = (1 - h_T^2)\left(h_{T-1} + w\frac{\partial h_{T-1}}{\partial w}\right).$$

به طور مشابه برای $\frac{\partial h_{T-1}}{\partial w}$ نیز داریم:

$$\frac{\partial h_{T-1}}{\partial w} = (1-h_{T-1}^2)\left(h_{T-2} + w\frac{\partial h_{T-2}}{\partial w}\right).$$

از سوی دیگر با توجه به $\frac{\partial h_0}{\partial w} = 0$ (ثابت بودن h_0) داریم:

$$\frac{\partial h_1}{\partial w} = (1-h_1^2)h_0.$$

با جایگذاری مشتقات جزئی در رابطهٔ اصلی و جمع روی همهٔ مسیرها داریم:

$$\begin{aligned}\frac{\partial h_T}{\partial w} &= (1-h_T^2)h_{T-1} \\ &+ (1-h_T^2)\,w\,(1-h_{T-1}^2)\,h_{T-2} \\ &+ (1-h_T^2)\,w\,(1-h_{T-1}^2)\,w\,(1-h_{T-2}^2)\,h_{T-3} \\ &+ \cdots \\ &+ (1-h_T^2)\,w\,(1-h_{T-1}^2)\cdots w\,(1-h_2^2)\cdot(1-h_1^2)\,h_0.\end{aligned}$$

پس از جابجایی جملات داریم:

$$\begin{aligned}\frac{\partial h_T}{\partial w} = \quad & w^0\,(1-h_T^2)h_{T-1} \\ &+ w^1\,(1-h_{T-1}^2)\,h_{T-2}\cdot(1-h_T^2) \\ &+ w^2\,(1-h_{T-2}^2)\,h_{T-3}\cdot(1-h_T^2)\,(1-h_{T-1}^2) \\ &+ \cdots \\ &+ w^{T-1}\,(1-h_1^2)\,h_0\cdot(1-h_T^2)\,(1-h_{T-1}^2)\cdots(1-h_2^2).\end{aligned}$$

با بازنویسی عبارت فوق داریم:

$$\frac{\partial h_T}{\partial w} = \sum_{t=1}^{T} w^{T-t}\cdot(1-h_t^2)h_{t-1}\cdot\prod_{j=t+1}^{T}(1-h_j^2)$$

که در آن فرض شده است حاصل‌ضرب برای اندیس‌های تهی برابر ۱ است.
در نتیجه حاصل گرادیان نهایی به شکل زیر خواهد بود:

$$\frac{\partial L}{\partial w} = (h_T - y^*)\sum_{t=1}^{T} w^{T-t}\cdot(1-h_t^2)h_{t-1}\cdot\prod_{j=t+1}^{T}(1-h_j^2)$$

بخش دوم

آ) با توجه به تعریف تابع tanh داریم $0 < 1-h_i^2 \leq 1$. پس داریم:

$$0 < |\gamma_i| = |w|(1-h_i^2) \leq |w|$$

ب) می‌دانیم

$$\prod_{i=t}^{T-1}\gamma_i = w^{T-t}\prod_{i=t}^{T-1}(1-h_i^2).$$

پس اگر $|w| < 1$ باشد، آنگاه داریم $|w|^{T-t} \to 0$. اگر $|w| = 1$ باشد و h_tها ناصفر باشند، داریم $\prod(1-h_i^2) \to 0$. بنابراین برای $|w| \leq 1$ (با فرض غیرصفر بودن h_tها) اضمحلال رخ می‌دهد.

ج) برای واگرایی نیاز است $|w| > 1$ باشد و همچنین $\prod_{i=t}^{T-1}(1-h_i^2)$ سریع‌تر از $|w|^{-(T-t)}$ کاهش نیابد. تنها در حالتی که به ازای همهٔ مقادیر t داشته باشیم $h_t = 0$، آنگاه خواهیم داشت

$$\prod_{i=t}^{T-1} \gamma_i = |w|^{-(T-t)} \to \infty.$$

در غیراین‌صورت تابع $\tanh$ موجب کوچک شدن $\prod_{i=t}^{T-1}(1-h_i^2)$ و جلوگیری از انفجار گرادیان می‌شود.

پرسش ۶. مقایسهٔ معماری‌های LSTM و GRU (۱۵ نمره)

در درس با دو معماری LSTM و GRU به تفصیل آشنا شده‌اید. در این پرسش قصد داریم این دو معماری را با یکدیگر مقایسه نماییم و نشان دهیم تحت فرضیاتی این دو مدل هم‌ارز خواهند بود. در ادامه برای یادآوری روابط این دو معماری آورده شده است.

مدل LSTM:

$$
\begin{aligned}
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{(گیت ورودی)} \\
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{(گیت فراموشی)} \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{(گیت خروجی)} \\
\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

مدل GRU:

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(گیت بروزرسانی)} \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(گیت بازنشانی)} \\
\tilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}
$$

بخش اول

فرض کنید در مدل LSTM، گیت خروجی همواره باز است یعنی $o_t = 1$ و مقدار h_t بدون تابع فعالسازی $\tanh$ محاسبه می‌شود. همچنین در نظر بگیرید گیت‌های ورودی و فراموشی مکمل یکدیگر رفتار می‌کنند یعنی $f_t = 1 - i_t$ است.

آ) با توجه به فرضیات فوق h_t را به عنوان تابعی از i_t ، x_t و h_{t-1} بازنویسی کنید.

ب) رابطهٔ بازنویسی‌شده برای h_t را با مدل GRU مقایسه کنید. شباهت و تفاوت آن دو را توضیح دهید.

بخش دوم

آ) نقش گیت بازنشانی r_t در مدل GRU را توصیف کنید و برای مدل LSTM تغییری در رابطهٔ $\tilde{c}_t$ بدهید به‌طوری که بتواند توانایی مشابه با گیت بازنشانی مدل GRU پیدا کند.

ب) با استفاده از نقش گیت‌ها در مدل GRU و مدل LSTM با تغییرات فوق، توضیح دهید هر کدام از فرضیات به چه معنایی هستند و شباهت این دو مدل را بررسی کنید.

پاسخ. ◁

بخش اول

آ) با فرض $o_t = 1$، $h_t = c_t$ و $f_t = 1 - i_t$:

$$h_t = c_t = f_t \odot h_{t-1} + i_t \odot \tilde{c}_t = (1 - i_t) \odot h_{t-1} + i_t \odot \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

ب) رابطهٔ بالا مشابه با مدلGRU با $r_t = 1$ است، جایی که i_t نقش گیت بروزرسانی z_t را بازی می‌کند. در مدل GRU کاندید به $r_t \odot h_{t-1}$ وابسته است که در اینجا وجود ندارد و در عوض از کل h_{t-1} استفاده می‌شود.

بخش دوم

آ) گیت بازنشانی r_t در مدل GRU مشخص می‌کند چه مقدار از حالت پنهان قبلی در محاسبهٔ کاندید نادیده گرفته شود. برای افزودن این قابلیت به مدل LSTM، ابتدا $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$ تعریف کرده و سپس کاندید را اصلاح می‌کنیم:

$$\tilde{c}_t = \tanh(W_c x_t + U_c(r_t \odot h_{t-1}) + b_c)$$

با حفظ $o_t = 1$، $h_t = c_t$، $f_t = 1 - i_t$، بروزرسانی h_t می‌شود:

$$h_t = (1 - i_t) \odot h_{t-1} + i_t \odot \tanh(W_c x_t + U_c(r_t \odot h_{t-1}) + b_c)$$

ب) فرض $o_t = 1$ و $h_t = c_t$ به معنای حذف گیت خروجی و ادغام حالت سلول و پنهان است. فرض $f_t = 1 - i_t$ دو گیت ورودی و فراموشی را در یک گیت بروزرسانی ترکیب می‌کند. افزودن r_t به کاندید، قابلیت بازنشانی GRU را به LSTM می‌دهد. بدین ترتیب با توجه به تساوی $z_t = i_t$ و تساوی پارامترها، دو مدل کاملاً هم‌ارز می‌شوند.

پرسش ۷. جریان گرادیان و معماری‌های کمینه RNN (۱۰ نمره)

قبلا بررسی کردید که چگونه شبکه‌های بازگشتی استاندارد از مشکل محو شدن گرادیان رنج می‌برند و چگونه معماری‌هایی مانند LSTM و GRU با استفاده از مکانیزم‌های گیت‌گذاری این مشکل را کاهش می‌دهند. در این پرسش، قصد داریم جریان گرادیان حالت سلول را در یک LSTM از نظر ریاضی تحلیل کنیم و سپس به بررسی یک نسخه سفارشی و بسیار ساده‌شده از GRU بپردازیم.

بخش اول: چرخ و فلک خطای ثابت در LSTM (Constant Error Carousel)

معادله به‌روزرسانی اصلی برای حالت سلول c_t در LSTM به صورت زیر است:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

که در آن f_t گیت فراموشی، i_t گیت ورودی، و $\tilde{c}_t$ حالت سلول کاندید است.

(الف) در فرمول‌بندی اصلی LSTM، برای جلوگیری از انفجار یا محو شدن گرادیان از طریق مسیرهای پیچیده، خطا بریده (Truncate) می‌شود. فرض کنید گرادیان‌های گیت‌ها (f_t, i_t) و حالت کاندید ($\tilde{c}_t$) نسبت به c_{t-1} برابر صفر در نظر گرفته شوند (یعنی ما تنها مسیر خطی مستقیم حالت سلول را در نظر می‌گیریم). تحت این فرض، گرادیان محلی $\frac{\partial c_t}{\partial c_{t-1}}$ را به دست آورید.

(ب) با استفاده از نتیجه بخش (الف)، گرادیان کلی $\frac{\partial c_T}{\partial c_t}$ را روی دنباله‌ای به طول T محاسبه کنید. از نظر ریاضی توضیح دهید که چگونه تنظیم بایاس گیت فراموشی b_f روی یک عدد مثبت بزرگ در زمان مقداردهی اولیه، به کاهش مشکل محو شدن گرادیان کمک می‌کند. این رفتار را با یک شبکه استاندارد RNN مقایسه کنید.

بخش دوم: واحد گیت‌دار کمینه

برای کاهش بار محاسباتی معماری GRU (که دارای دو گیت به‌روزرسانی z_t و بازنشانی r_t است)، محققان معماری **واحد گیت‌دار کمینه (MGU)** را پیشنهاد دادند که گیت‌های بازنشانی و به‌روزرسانی را در یک گیت فراموشی f_t ترکیب می‌کند. MGU به صورت زیر تعریف می‌شود:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(f_t \odot h_{t-1}) + b_h)$$
$$h_t = (1 - f_t) \odot h_{t-1} + f_t \odot \tilde{h}_t$$

(الف) معادلات MGU را با GRU استاندارد مقایسه کنید. به‌طور مشخص، گیت‌های GRU استاندارد (z_t و r_t) را بر حسب گیت f_t در MGU بیان کنید تا نشان دهید چگونه MGU حالت خاصی از معماری GRU است.

(ب) رفتار دقیق MGU را در مقادیر حدی تابع فعال‌ساز سیگموید تعیین کنید:

- **حالت ۱:** اگر برای تمام ابعاد $f_t \to 1$ میل کند چه اتفاقی می‌افتد؟ در این حالت، MGU دقیقاً رفتار کدام معماری کلاسیک را تقلید می‌کند؟
- **حالت ۲:** اگر برای تمام ابعاد $f_t \to 0$ میل کند چه اتفاقی می‌افتد؟ کاربرد عملی این حالت برای مدل‌سازی توالی‌ها چیست؟

پاسخ. ◁

بخش اول: چرخ و فلک خطای ثابت در LSTM

(الف) استخراج گرادیان محلی:

با توجه به معادله حالت سلول:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

به ما گفته شده است که گرادیان‌های عبوری از توابع غیرخطی گیت‌ها را نادیده بگیریم (بریده شوند). این بدان معناست که فرض می‌کنیم $\frac{\partial f_t}{\partial c_{t-1}} = 0$، $\frac{\partial i_t}{\partial c_{t-1}} = 0$ و $\frac{\partial \tilde{c}_t}{\partial c_{t-1}} = 0$.

مشتق‌گیری از c_t نسبت به c_{t-1} تحت این شرایط، تنها اتصال خطی مستقیم را باقی می‌گذارد:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t)$$

که در آن $\text{diag}(f_t)$ یک ماتریس قطری است که عناصر بردار f_t روی قطر اصلی آن قرار دارند.

(ب) گرادیان کلی و مقداردهی اولیه:

با استفاده از قاعده زنجیره‌ای و نتیجه بخش (الف)، گرادیان کلیِ حالت سلول از گام T به گام t برابر با حاصل‌ضرب گرادیان‌های محلی است:

$$\frac{\partial c_T}{\partial c_t} = \prod_{k=t+1}^{T} \frac{\partial c_k}{\partial c_{k-1}} = \prod_{k=t+1}^{T} \text{diag}(f_k)$$

اگر بایاس گیت فراموشی (b_f) را با یک مقدار مثبت بزرگ مقداردهی اولیه کنیم، تابع سیگموید $\sigma(\cdot)$ در مقادیری نزدیک به ۱ اشباع می‌شود. بنابراین، $f_k \approx \mathbf{1}$ (یک بردار از یک‌ها). با جایگذاری این تقریب در گرادیان کلی خواهیم داشت:

$$\frac{\partial c_T}{\partial c_t} \approx \prod_{k=t+1}^{T} I = I$$

این نشان می‌دهد که گرادیان $\frac{\partial c_T}{\partial c_t}$ نزدیک به ۱ (ماتریس همانی) باقی می‌ماند و به طور کامل از فروپاشی نمایی جلوگیری می‌کند. به این مکانیزم چرخ‌فلک خطای ثابت (Constant Error Carousel) یا CEC می‌گویند.

در مقابل، پس‌انتشار در یک شبکه Vanilla RNN حاصل‌ضربی شامل ماتریس وزن W و مشتق تابع $\tanh$ تولید می‌کند: $\prod W^T \text{diag}(1 - h_k^2)$. از آنجا که نرم این حاصل‌ضرب معمولاً کمتر از ۱ است، گرادیان در توالی‌های طولانی به صورت نمایی محو می‌شود. معماری LSTM با هدایت جمعیِ (additive) گرادیان‌ها از طریق حالت سلول، این مشکل را دور می‌زند.

بخش دوم: واحد گیت‌دار کمینه (MGU)

(الف) مقایسه ساختاری با GRU:

معادلات GRU استاندارد را به یاد بیاورید:

$$\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\
\tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}$$

با مقایسه این روابط با معادلات MGU، می‌توانیم ببینیم که MGU گیت به‌روزرسانی z_t و گیت بازنشانی r_t را دقیقاً به یکدیگر پیوند داده و یکی می‌کند. به طور خاص، MGU معادل یک GRU است که در آن:

$$z_t = f_t$$

$$r_t = f_t$$

در معماری MGU، تک‌گیت f_t هم‌زمان تعیین می‌کند که هنگام محاسبه کاندید $\tilde{h}_t$ چه مقدار از حالت قبلی بازنشانی شود و همچنین چه مقدار از کاندید در حالت پنهان نهایی h_t «به‌روزرسانی» گردد.

(ب) تحلیل در مقادیر حدی:

حالت ۱ ($f_t \to 1$):

اگر $f_t = 1$، معادلات به این شکل درمی‌آیند:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(\mathbf{1} \odot h_{t-1}) + b_h) = \tanh(W_h x_t + U_h h_{t-1} + b_h)$$
$$h_t = (1 - 1) \odot h_{t-1} + 1 \odot \tilde{h}_t = \tilde{h}_t$$

بنابراین، $h_t = \tanh(W_h x_t + U_h h_{t-1} + b_h)$.

نتیجه‌گیری: در این حالت، MGU کاملاً رفتار یک شبکه **RNN** استاندارد را تقلید می‌کند و حافظه خود را به طور کامل با حالت کاندید جدید بازنویسی می‌نماید.

حالت ۲ ($f_t \to 0$):

اگر $f_t = 0$، معادلات به این شکل درمی‌آیند:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(\mathbf{0} \odot h_{t-1}) + b_h) = \tanh(W_h x_t + b_h)$$
$$h_t = (1 - 0) \odot h_{t-1} + 0 \odot \tilde{h}_t = h_{t-1}$$

نتیجه‌گیری: در این حالت، $h_t = h_{t-1}$ خواهد بود. شبکه به طور کامل ورودی فعلی x_t و حالت کاندید را نادیده می‌گیرد. مفهوم و کاربرد عملی آن این است که شبکه به عنوان یک **سلول حافظه خالص** عمل می‌کند و اطلاعات گذشته را بدون هیچ تغییری به جلو انتقال می‌دهد. این ویژگی برای پر کردن شکاف‌های زمانی طولانی و حفظ اطلاعات در فواصل دور توالی بسیار مفید است.